



Chapter S4E: Socket Programming

CONTENT

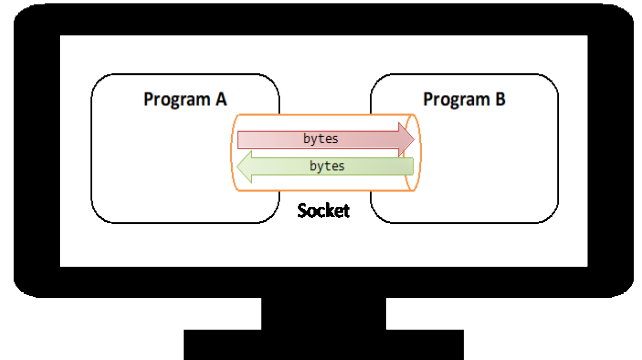
No.	Topic	Syllabus Guide
1	Sharing Data Between Programs	4.1.6
2	Creating a Socket Connection	
3	Using the socket Module	
4	Designing a Protocol	
5	Iterative Server	
6	Writing a Chat Program	
7	Writing a Symmetrical Turn-Based Game	
8	Writing an Asymmetrical Game	
9	Quiz	
10	References	
11	Programming Exercise	



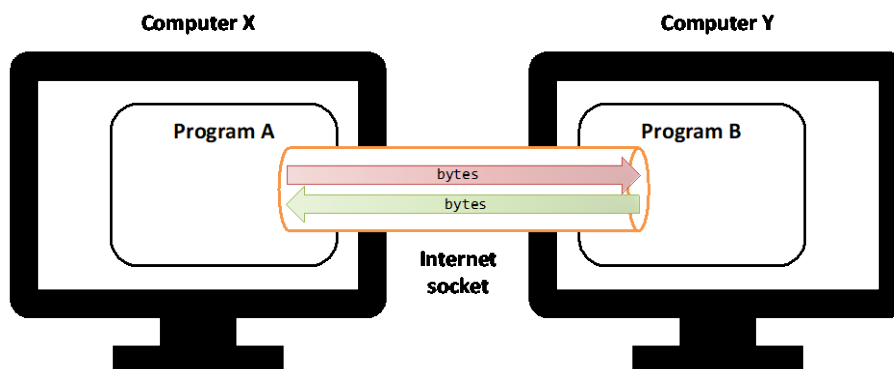
1. Sharing Data Between Programs

Suppose two Python programs are running at the same time. How to send data from one program to the other and vice versa? Most operating systems provide a mechanism called **sockets** to do this.

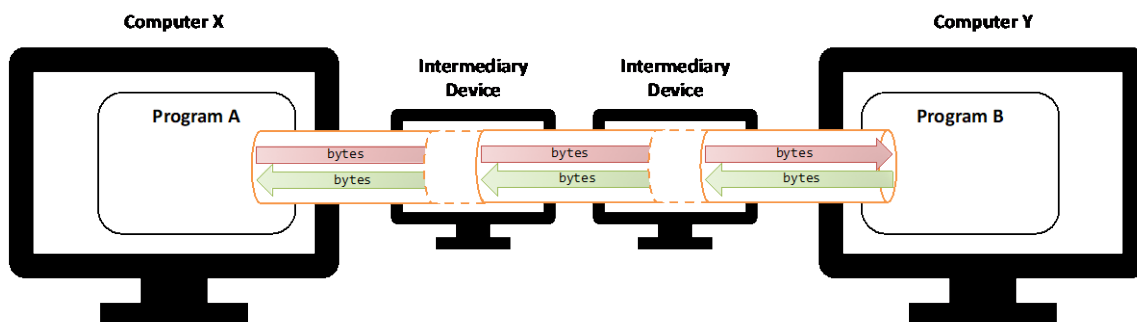
A socket connection is like a pipe between two running programs. The pipe is bidirectional and can carry data (in bytes) in both directions.



There are many kinds of sockets, but the kind that is most often discussed is called an **Internet socket**. Internally, Internet sockets deliver data using the same Transmission Control Protocol and Internet Protocol suite (commonly abbreviated as TCP/IP) that is used to transmit data over the Internet. This means that Internet sockets can deliver data between *any* two programs, even programs that are running on different computers, as long as the two computers can access each other over the network.

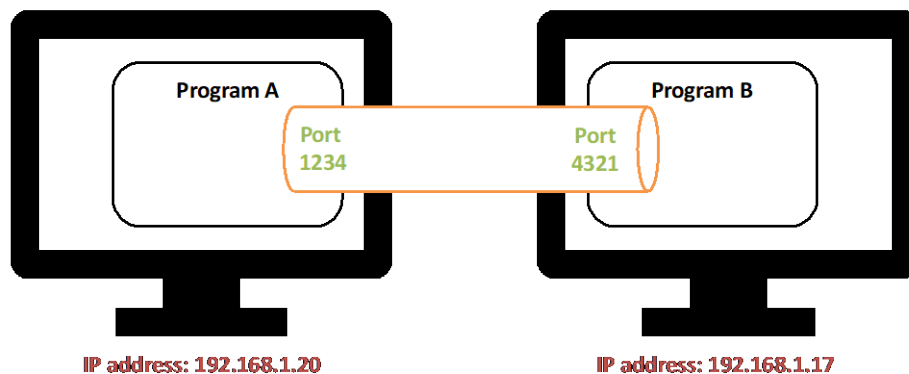


For simplicity, we illustrate an Internet socket as a pipe that is only attached to the two computers. In reality, however, data that is transmitted through an Internet socket may pass through multiple devices before reaching its destination. You should be aware that any of these devices can steal or modify the data that passes through a socket unless you encrypt the data first. A more accurate illustration of a socket that shows how the data passes through multiple devices is shown below:





Each end of a socket is associated with a running program and is uniquely identified by an **IP address** and **port number**. The IP address identifies which device that end of the socket is attached to and the port number identifies which program on that device is using the socket.



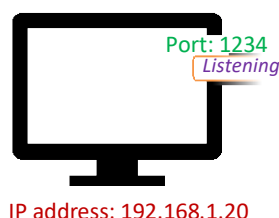
On a Windows computer, you can run the command `netstat -n` to list out the sockets that are currently open on your computer. Each socket will be displayed with a combined IP address and port number for each of its ends (i.e., its local and foreign addresses):

To reveal which program is using each socket, you can run `netstat -no` to reveal the process ID (PID) associated with each socket. You can then open **Task Manager** and match each PID to the name of a running program.

2. Creating a Socket Connection

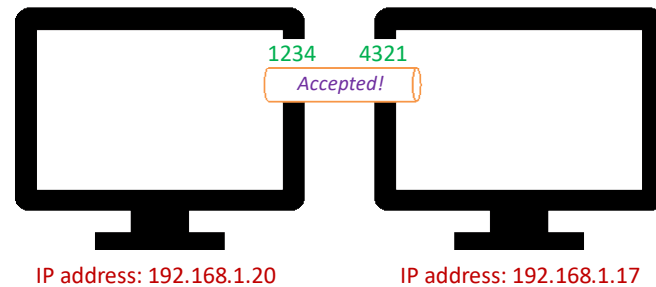
Creating a socket connection is a multi-step process that requires one program to be the **server** and another program to be the **client**. The server's IP address and port number for accepting connections must also be known ahead of time by the client.

First, the server creates a **passive socket**, binds it to the pre-chosen port number and listens for an incoming connection. (A passive socket is not connected and merely waits for an incoming connection.)

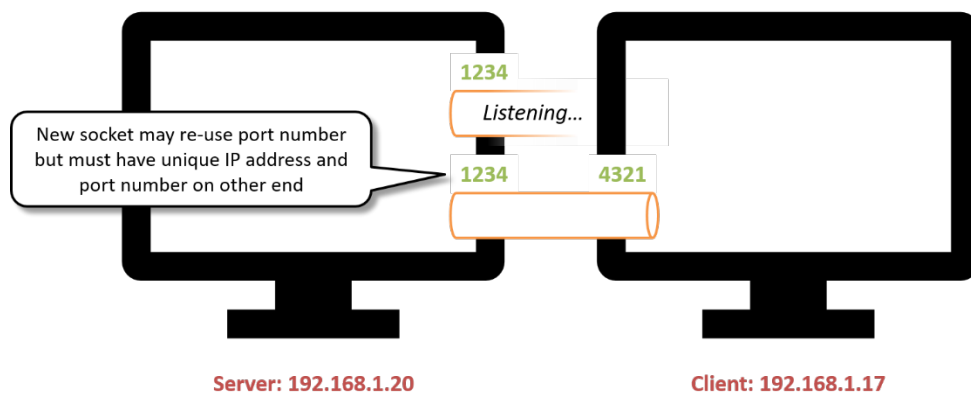


Next, the client initiates a connection request using the server's IP address and port number. If no server is listening on the chosen port, the connection will be refused.

On the other hand, if the connection request reaches an IP address and port number that a server is listening on, the server accepts and creates a new socket for the requesting client using a dynamically assigned port number.



The passive socket goes back to listening for new connections while the client and server can now exchange data using the newly-created socket.



Note that the newly-created socket is symmetrical: data sent on one end is received on the other end and vice versa. Once a socket is established, it can send data *both* from the client to the server *and* from the server to the client.

3. Using the socket Module

You can create and manage sockets in Python by importing the **socket** module and creating **socket** objects. The methods of the socket class are summarised below:

Methods	Description
bind((host, port))	Binds socket object to the given address tuple (host, port), where host is an IPv4 address and port is a port number
listen()	Enables socket to listen for incoming connections from clients
accept()	Waits for an incoming connection and returns a tuple containing a new socket object for the connection and an address tuple (host, port), where host is the IPv4 address of the connected client and port is its port number



<code>connect ((host, port))</code>	Initiates a connection to the given address tuple (host, port), where host is the IPv4 address of the server and port is its port number
<code>recv (max_bytes)</code>	Receives and returns up to the given number of bytes from the socket
<code>sendall (bytes)</code>	Sends the given bytes to the socket

Create the following **basic server** program that listens for a client on port 12345, accepts a connection request, sends `b'Hello from server\n'` to the client through the socket, then closes the socket.

Program 1: `basic_server.py`

```
1  import socket
2
3  my_socket = socket.socket()
4  my_socket.bind(('127.0.0.1', 12345))
5  my_socket.listen()
6
7  new_socket, addr = my_socket.accept()
8  print('Connected to: ' + str(addr))
9  new_socket.sendall(b'Hello from server\n')
10 new_socket.close()
11 my_socket.close()
```

Note that instead of 12345 on line 4, we could have chosen any large port number to use. This number must be decided ahead of time for the client (written later) to use when connecting.

On line 7, `socket.accept()` returns a tuple of the newly created socket and a nested address tuple. We store both the new socket and the address tuple in two variables named `new_socket` and `addr` respectively. Note that `new_socket` is the socket that we actually use to send and receive data.

Run this program. If everything is working correctly, the server should appear stuck shortly after it is started. This is because the `socket.accept()` method is **blocking**¹ the program and prevents it from continuing until a connection request is received.

To create a client that can connect to this server, start a second copy of Python. Create a new Python program using the *second* copy of Python.

¹ A “blocked” process means that it is waiting for some event to occur.



In the window that appears, enter the following **basic client** program that asks for the server's IP address and port number, requests for a connection, receives and prints at most 1024 bytes from the server, then closes the socket.

Program 2: `basic_client.py`

```
1 import socket
2
3 my_socket = socket.socket()
4
5 address = input('Enter IPv4 address of server: ')
6 port = int(input('Enter port number of server: '))
7
8 my_socket.connect((address, port))
9 print(my_socket.recv(1024))
10 my_socket.close()
```

Note that the argument for `socket.recv()` is required and should be set to a relatively small power of 2. In this case, we use a value of 2^{10} or 1024.

Run this program using the second copy of Python and make sure the server you started previously is still running.

At this point, the client should be prompting you for the address and port number of the server. Use the special IPv4 address **127.0.0.1** that refers to the local machine and enter **12345** as the port number. The client should successfully connect to the server and print out the bytes that were received. At the same time, the server program should become unstuck and end normally.



Practice 1: Transfer data in both directions

Recall that once a socket is established, it is symmetrical and can transfer data in both directions. The basic server and client program above only send data in one direction. Now, write your own server and client to demonstrate that data can be sent in the opposite direction. Specifically, the client should send `b'Hello from client\n'` to the server and the server should print out any bytes that are received from the client.



4. Designing a Protocol

As networks can become congested, we cannot assume that data sent over Internet sockets will be transmitted instantaneously. For instance, if a message is too long to fit into a packet, a program may receive only the first half of a message before the second half arrives some time later. To avoid working with incomplete data, we will need to define a **protocol** so that the start and end of messages can be detected unambiguously.

The only way to be certain that any received data is complete is to agree beforehand on a **protocol** or set of rules for how communication should take place. For instance, we can agree beforehand that any data we transmit will always end with a newline character `\n` and that the data itself will never contain the `\n` character. This very simple protocol allows us to detect the end of a transmission easily by just searching for the `\n` character.

The following projects updates the client so that it uses the `\n` character to detect when the message ends. This new client calls `socket.recv()` continuously and appends the received bytes to a variable named `data` until the `\n` character is encountered.

Program 3: `basic_client_protocol.py`

```
1  import socket
2
3  my_socket = socket.socket()
4
5  address = input('Enter IPv4 address of server: ')
6  port = int(input('Enter port number of server: '))
7
8  my_socket.connect((address, port))
9  data = b''
10 while b'\n' not in data:
11     data += my_socket.recv(1024)
12 print(data)
13 my_socket.close()
```

With this new client, all the data sent by the server up to and including the `\n` character is successfully received and printed.



5. Iterative Server

Currently, the server program exits immediately after it finishes working with a client. In reality, we often want the server program to run continuously so that it is always listening and available for multiple clients to send connection requests. We can do this by putting the code that deals with a client in an infinite loop.

Program 4: `basic_server_iterative.py`

```
1  import socket
2
3  my_socket = socket.socket()
4  my_socket.bind(('127.0.0.1', 12345))
5  my_socket.listen()
6
7  while True:
8      new_socket, addr = my_socket.accept()
9      new_socket.sendall(b'Hello from server\n')
10     new_socket.close()
```

To interrupt a program that is running in an infinite loop, press **Ctrl-C**. In IDLE, we can also restart the shell using **Ctrl-F6**.

Internally, the server's passive socket keeps a queue of connection requests that have been received. A request is removed from this queue each time `socket.accept()` is called to create a connection. (If the queue is empty, `socket.accept()` will block the program until a connection request is received, as expected.)

Since `socket.accept()` is called each time the infinite loop repeats, our program is able to handle multiple clients by processing them one at a time. This means that our program works as an **iterative server**. Iterative servers are easy to write but limited as they can only handle one client at a time.

Alternatively, we could have written our server such that it starts a **thread** that runs simultaneously with the main program each time a client tries to connect. This makes the program more complicated to write but will let it to handle multiple clients at the same time, hence making it a **concurrent server**. In our course, we only focus on an iterative server.



6. Writing a Chat Program

We now have all the tools needed to write a simple chat client and server such that two users can take turns sending single lines of text to each other. One user would be running the server and the other user would be running the client.

Since each message is restricted to a single line, we can be certain that the newline character `\n` will never be part of a message. This means that we can adopt a similar protocol of using `\n` to detect the end of a message.

Let us use a different port number of 6789 and create the following chat server program that repeatedly prompts the user for some text, sends that text to the client (after encoding it into bytes), then receives and prints out the client's response.

Program 5: chat_server.py

```
1  import socket
2
3  listen_socket = socket.socket()
4  listen_socket.bind(('127.0.0.1', 6789))
5  listen_socket.listen()
6
7  chat_socket, addr = listen_socket.accept()
8  while True:
9      data = input('INPUT SERVER: ').encode()
10     chat_socket.sendall(data + b'\n')
11     print('WAITING FOR CLIENT...')
12     data = b''
13     while b'\n' not in data:
14         data += chat_socket.recv(1024)
15     print('CLIENT WROTE: ' + data.decode())
```

The client program is similar, except the order of sending and receiving is reversed.

Program 6: chat_client.py

```
1  import socket
2
3  chat_socket = socket.socket()
4
5  address = input('Enter IPv4 address of server: ')
6  port = int(input('Enter port number of server: '))
7
```



```
8 chat_socket.connect((address, port))
9 while True:
10     print('WAITING FOR SERVER...')
11     data = b''
12     while b'\n' not in data:
13         data += chat_socket.recv(1024)
14     print('SERVER WROTE: ' + data.decode())
15     data = input('INPUT CLIENT: ').encode()
16     chat_socket.sendall(data + b'\n')
```

Run the server and client using two different copies of Python. Once again, since the server is running on the same machine as the client, we can use 127.0.0.1 as the server's IPv4 address and 6789 as the port number.



Practice 2: Allow users to 'Quit' programs

Currently, there is no way to exit our chat programs other than to press Ctrl-C or to restart the shell (in IDLE).

Modify `chat_server.py` and `chat_client.py` so that both programs exit once the message 'quit' is sent by any user. Remember to make sure that all sockets are closed properly before exiting.

7. Writing a Symmetrical Turn-Based Game

So far, we have been responsible for writing both the server and client programs. Sometimes, however, both server and protocols designs may be based on an existing standard or developed by someone else. To write a client that can communicate with an existing server, we need to study its code and follow the expected protocol.

Conversely, sometimes the client may be developed by someone else and we need to write a server to communicate with it. In either case, it is important to start by understanding the protocol being used.

To demonstrate how to do this, let us examine the server program for a simple symmetrical turn-based 2-player game of Tic-Tac-Toe. Both players have the same goal of trying to get three of their symbols in a row, column, or diagonal on the game board. First, we create a simple library that defines some constants and a `TicTacToe` class to handle the game logic:



Program 7: tictactoe.py

```
1  N = 3                                # Size of grid
2  WIDTH = len(str(N ** 2))             # Width for each cell
3  PLAYERS = ('O', 'X')                 # Player symbols
4
5  class TicTacToe:
6
7      def __init__(self):
8          self.board = []
9          for i in range(N):
10             self.board.append([None] * N)
11
12     def render_row(self, row_index):
13         start = row_index * N + 1
14         row = self.board[row_index].copy()
15         for column_index in range(N):
16             if row[column_index] is None:
17                 cell = str(start + column_index)
18             else:
19                 cell = PLAYERS[row[column_index]]
20             if len(cell) < WIDTH:
21                 cell += ' ' * (WIDTH - len(cell))
22             row[column_index] = ' ' + cell + ' '
23         return '|'.join(row) + '\n'
24
25     def render_board(self):
26         rows = []
27         for row_index in range(N):
28             rows.append(self.render_row(row_index))
29         divider = '-' * ((WIDTH + 3) * N - 1) + '\n'
30         return divider.join(rows)
31
32     def make_move(self, player_index, cell_index):
33         cell_index -= 1
34         self.board[cell_index // N][
35             cell_index % N] = player_index
36
37     def is_valid_move(self, cell_index):
38         if cell_index < 1 or cell_index > N ** 2:
39             return False
40         cell_index -= 1
41         return self.board[cell_index // N][
42             cell_index % N] is None
```



```
43
44     def is_full(self):
45         for row_index in range(N):
46             for column_index in range(N):
47                 if self.board[row_index][
48                     column_index] is None:
49                     return False
50         return True
51
52     def get_winner(self):
53         # Check diagonals
54         if self.board[0][0] is not None:
55             found = True
56             for i in range(N):
57                 if self.board[0][0] != self.board[i][i]:
58                     found = False
59                     break
60             if found:
61                 return self.board[0][0]
62         if self.board[0][N - 1] is not None:
63             found = True
64             for i in range(N):
65                 if self.board[0][N - 1] != self.board[i][
66                     N - i - 1]:
67                     found = False
68                     break
69             if found:
70                 return self.board[0][N - 1]
71
72         # Check rows and columns
73         for i in range(N):
74             if self.board[i][0] is not None:
75                 found = True
76                 for j in range(N):
77                     if self.board[i][0] != self.board[i][j]:
78                         found = False
79                         break
80                 if found:
81                     return self.board[i][0]
82             if self.board[0][i] is not None:
83                 found = True
84                 for j in range(N):
85                     if self.board[0][i] != self.board[j][i]:
86                         found = False
```



87	<code>break</code>
88	<code>if found:</code>
89	<code>return self.board[0][i]</code>
90	
91	<code># No matching lines were found, so no winner</code>
92	<code>return None</code>

The following is a summary of the methods in **TicTacToe** class:

Methods	Description
<code>render_row(row_index)</code>	Returns a string representation of the specified row, such as: <code>1 2 3</code>
<code>render_board()</code>	Returns a string representation of the entire board, such as: <code>1 2 3</code> <code>-----</code> <code>4 5 6</code> <code>-----</code> <code>7 8 9</code>
<code>make_move(player_index, cell_index)</code>	Modifies the board such that the specified cell is marked with the symbol for the specified player
<code>is_valid_move(cell_index)</code>	Returns whether the specified cell is currently blank
<code>is_full()</code>	Returns whether the entire board has been filled up
<code>get_winner()</code>	Returns winning player for the current board or None if there is no winner

Using this library, we create a server program that creates a **TicTacToe** object on line 9 to store information about the Tic-Tac-Toe board:

Program 8: <code>game_server.py</code>	
1	<code>import socket</code>
2	<code>import tictactoe</code>
3	
4	<code>listen_socket = socket.socket()</code>
5	<code>listen_socket.bind(('127.0.0.1', 3456))</code>
6	<code>listen_socket.listen()</code>
7	
8	<code>game_socket, addr = listen_socket.accept()</code>



```
9  game = tictactoe.TicTacToe()
10 while True:
11     # Display current Tic-Tac-Toe board
12     print(game.render_board())
13
14     # Check if client player won
15     if game.get_winner() is not None:
16         print('Opponent wins!')
17         print()
18         break
19
20     # Check if board is full
21     if game.is_full():
22         print('Stalemate')
23         print()
24         break
25
26     # Prompt for move from server player
27     move = -1
28     while move != 0 and not game.is_valid_move(move):
29         move = int(input('Server moves ' +
30                         '(0 to quit): '))
31     print()
32     if move == 0:
33         game_socket.sendall(b'END\n')
34         print('You quit, opponent wins!')
35         print()
36         break
37     game.make_move(0, move)
38     game_socket.sendall(b'MOVE' +
39                         str(move).encode() + b'\n')
40
41     # Display current Tic-Tac-Toe board
42     print(game.render_board())
43
44     # Check if server player won
45     if game.get_winner() is not None:
46         print('You win!')
47         print()
48         break
49
50     # Check if board is full
51     if game.is_full():
52         print('Stalemate')
```



```
53         print()
54         break
55
56     # Receive move from client player
57     received = b''
58     while b'\n' not in received:
59         received += game_socket.recv(1024)
60     if received.startswith(b'MOVE') :
61         move = int(received[4:])
62         print('Client moves: ' + str(move))
63         print()
64         game.make_move(1, move)
65     elif received.startswith(b'END') :
66         print('Opponent quits, you win!')
67         print()
68         break
69
70 game_socket.close()
71 listen_socket.close()
```

Analysing this server code, we see that communications with the client is divided into several steps that repeat in an infinite loop:

1. Display current Tic-Tac-Toe board
2. Check if opponent has won, and if so, end game with opponent winning
3. Check if the board is full, and if so, end game with a stalemate
4. Prompt for input from player; if player makes a valid move, update game board accordingly, then send **b'MOVE'** followed by the chosen cell number and **b'\n'** to the opponent; if player chooses to quit, send **b'END\n'** to the opponent and end game with the opponent winning
5. Display current Tic-Tac-Toe board again
6. Check if player has won, and if so, end game with player winning
7. Check if the board is full, and if so, end game with a stalemate
8. Receive opponent's action via the socket; if the action is **b'MOVE'** followed by a cell number and **b'\n'**, update game board accordingly; if the action is **b'END\n'**, end game with the player winning

As written, the server player always starts first. This means that our client code should start by receiving and processing the server's result. We also know that Tic-Tac-Toe is a symmetrical game (other than the choice of starting player), so we deduce that the client code should be similar to the server code except that "client" and "server" are exchanged and the last step is moved to the front:



1. Receive opponent's action via the socket; if the action is **b'MOVE'** followed by a cell number and **b'\n'**, update game board accordingly; if the action is **b'END\n'**, end game with the player winning
2. Display current Tic-Tac-Toe board
3. Check if opponent has won, and if so, end game with opponent winning
4. Check if the board is full, and if so, end game with a stalemate
5. Prompt for input from player; if player makes a valid move, update game board accordingly, then send **b'MOVE'** followed by the chosen cell number and **b'\n'** to the opponent; if player chooses to quit, send **b'END\n'** to the opponent and end game with the opponent winning
6. Display current Tic-Tac-Toe board again
7. Check if player has won, and if so, end game with player winning
8. Check if the board is full, and if so, end game with a stalemate

A client program that does this is as follows:

Program 9: `game_client.py`

```
1  import socket
2  import tictactoe
3
4  game_socket = socket.socket()
5  game_socket.connect(('127.0.0.1', 3456))
6
7  game = tictactoe.TicTacToe()
8  while True:
9      # Receive move from server player
10     received = b''
11     while b'\n' not in received:
12         received += game_socket.recv(1024)
13     if received.startswith(b'MOVE'):
14         move = int(received[4:])
15         print('Server moves: ' + str(move))
16         print()
17         game.make_move(0, move)
18     elif received.startswith(b'END'):
19         print('Opponent quits, you win!')
20         print()
21         break
22
23     # Display current Tic-Tac-Toe board
24     print(game.render_board())
25
26     # Check if server player won
27     if game.get_winner() is not None:
```




```
28         print('Opponent wins!')
29         print()
30         break
31
32     # Check if board is full
33     if game.is_full():
34         print('Stalemate')
35         print()
36         break
37
38     # Prompt for move from client player
39     move = -1
40     while move != 0 and not game.is_valid_move(move):
41         move = int(input('Client moves ' +
42                         '(0 to quit): '))
43     print()
44     if move == 0:
45         game_socket.sendall(b'END\n')
46         print('You quit, opponent wins!')
47         print()
48         break
49     game.make_move(1, move)
50     game_socket.sendall(b'MOVE' +
51                        str(move).encode() + b'\n')
52
53     # Display current Tic-Tac-Toe board
54     print(game.render_board())
55
56     # Check if client player won
57     if game.get_winner() is not None:
58         print('You win!')
59         print()
60         break
61
62     # Check if board is full
63     if game.is_full():
64         print('Stalemate')
65         print()
66         break
67
68     game_socket.close()
```

Run the server and client using two different copies of Python on the same machine to verify that the game works as expected.



Practice 3: Client starts game

Currently, the Tic-Tac-Toe game is written such that the server takes its turn first. Modify `game_server.py` and `game_client.py` so that the client takes its turn first instead.

8. Writing an Asymmetrical Game

While Tic-Tac-Toe is a symmetrical game where the rules are the same for both players, other games may be asymmetrical and thus require the two players to behave differently from each other.

The following is a client program for an asymmetric guess-the-number game where a server generates a random number from 1 to 100 and a client tries to guess it within 5 tries. After each incorrect guess, the server returns whether the guess is greater than or less than the required number:

Program 10: `guess_client.py`

```
1  import socket
2
3  s = socket.socket()
4  s.connect(('127.0.0.1', 9999))
5
6  data = b''
7  while True:
8      while b'\n' not in data:
9          data += s.recv(1024)
10         received = data[:data.find(b'\n')]
11         data = data[len(received) + 1:]
12         if received == b'LOW':
13             print('Your guess is too low.')
14         elif received == b'HIGH':
15             print('Your guess is too high.')
16         elif received == b'GUESS':
17             guess = int(input('Enter guess (1-100): '))
18             s.sendall(str(guess).encode() + b'\n')
19         elif received == b'WIN':
20             print('You win!')
21             break
22         elif received == b'GAMEOVER':
23             print('You ran out of tries! Game over.')
24             break
25
26  s.close()
```



Practice 4: Write the corresponding server program.



9. Quiz

1 Which of the following methods for sending data from one Python program to another does NOT work?

- A One program copies the data onto the system clipboard and the other program reads it from the system clipboard
- B One program writes the data into a file (that is readable to everyone) and the other program reads it from the same file
- C One programs assigns the data to a Python variable and the other program reads it from the same variable
- D The two programs set up at socket connection and transmit the data from one program to the other through the socket

()

2 Which of the following statements is true?

- A We can uniquely identify a program and which machine it is running on using only an IPv4 address.
- B We can uniquely identify a program and which machine it is running on using only an IPv4 or IPv6 address.
- C We can uniquely identify a program and which machine it is running on using only a port number.
- D We can uniquely identify a program and which machine it is running on using only an IPv4 or IPV6 address and a port number.

()

3 Which of the following statements about using sockets is FALSE?

- A Before connecting, the client must know the server's IP address and port number but not vice versa.
- B For each connection, the client's and server's port numbers must match.
- C The server must be running before the client can successfully connect.
- D The server uses a socket solely to listen for connection requests and creates a completely new socket each time it accepts a connection.

()

4 What is the role of the recv() method in socket programming?

- A It sends data from the client to the server.
- B It listens for incoming connections.
- C It accepts incoming connections.
- D It receives data from the socket.

()



5 Which method of the socket class is used to initiate a connection to a given address?

- A `bind((host, port))`
- B `listen()`
- C `accept()`
- D `connect((host, port))`

()

10. References

Python socket reference: <https://docs.python.org/3/library/socket.html>

Socket in Computer Network: <https://www.geeksforgeeks.org/socket-in-computer-network/>

socket Module Summary

Methods	Description
<code>bind((host, port))</code>	Binds socket object to the given address tuple (host, port), where host is an IPv4 address and port is a port number
<code>listen()</code>	Enables socket to listen for incoming connections from clients
<code>accept()</code>	Waits for an incoming connection and returns a tuple containing a new socket object for the connection and an address tuple (host, port), where host is the IPv4 address of the connected client and port is its port number
<code>connect((host, port))</code>	Initiates a connection to the given address tuple (host, port), where host is the IPv4 address of the server and port is its port number
<code>recv(max_bytes)</code>	Receives and returns up to the given number of bytes from the socket
<code>sendall(bytes)</code>	Sends the given bytes to the socket



11. Programming Exercise

Task 1. Hangman (Adapted from RVHS prelim 2020 P2/Q3)

The task is to write the client code of the game hangman with the help of server code given to you in "hangman_server.py". The server IPV4 address and its port used should be "127.0.0.1" and 12345.

The client program should also display the following menu as it runs:

Menu :

- 1) **Guess a letter**
- 2) **Guess a word**
- 3) **Quit**

Option 1 allows client to guess 1 letter a time

Option 2 allows client to guess the full word

Option 3 allows the client to quit the program

Study the server code and protocol before you write the client code.

Note: When you run the server code, you will be prompted to set a hidden word first.

Message	Client	Server
START	No message is required to be sent back to the server upon receiving the START message. However, it will print to terminal the following: Current word guessed: ?????? where the number of "?" is the length of the hidden word.	The START message is the FIRST message sent by the server to the client to inform the client the length of the hidden word. The following format is used: "START,<length_of_hidden_word>\n" For example, "START, 6\n"
GUESS	The GUESS message is sent to the server to make a guess of a letter of the hidden word. The following format is used: "GUESS,<letter>\n"	When the server receives a GUESS message, it sends a GUESS message back to the client to inform the client on the positions of the letter guessed correctly. For example, if the hidden word is "misses" and the following GUESS



	For example, to guess a "s", the following is used: "GUESS , s\n" Note: If the hidden word is not completely guessed, the server returns a GUESS message after a GUESS message is sent to the server. The GUESS message from the server will include the positions of the correctly guessed letter. Upon receiving the server's GUESS message, the client will print out the partially guessed word. For example, if s and i are guessed previously, the following should be printed to terminal. Current word guessed: ?iss?s Note: If the hidden word is completely guessed, the server returns a WIN message instead.	message is received from the client: "GUESS , s\n" The GUESS message returned from the server to the client will be: "GUESS , 2 , 3 , 5\n" If the letter guessed is not found in the hidden word, the following GUESS message is sent instead: "GUESS\n" If the hidden word is completely guessed, the server returns a WIN message. The format is as follow: "WIN\n"
HWORD	The HWORD message is sent to the server to guess the whole hidden word. The following format is used: "HWORD , <guess_word>\n" For example, if the client wants to guess "kisses", the following is sent: "HWORD , kisses\n"	When the server receives a HWORD message, it sends a WIN or LOSE message based on the <guess_word> and the hidden word. For example, if the hidden word is "misses" and the client guessed "kisses", the following LOSE message is sent: "LOSE\n"



	Note: The server will either return a WIN or LOSE message after a HWORD message is sent to the server. Upon receiving a WIN or LOSE message, the client will print the result to terminal, close its socket and quit.	If the hidden word is “misses” and the client also guessed “misses”, the following WIN message is sent: "WIN\n" Then, the server program closes all sockets and quits.
WIN / LOSE	No message is required to be sent back to the server upon receiving a WIN/LOSE message. The client just needs to print to terminal the result, closes all sockets and quits the program.	The WIN message is sent from the server to client, to inform the client has won. The format is as follow: "WIN\n" The LOSE message is sent from the server to client, to inform the client has lost. The format is as follow: "LOSE\n" The server then closes all sockets and quits the program after a WIN or LOSE message is sent to client.
QUIT	The QUIT message is sent to the server to close the connection. The following format is used: "QUIT\n" After the QUIT message is sent, the client will close its socket and the program quits.	When the server receives a QUIT message, the server then closes all sockets and quits. No message is sent back to the client.

Work in pairs to validate each other's programs. One of you will act as the server host and the other as client. Verify the following:

1. When the server sets a word, the client immediately displays the correct number of ?.
2. Guess a letter that appears twice. Does the client update both positions correctly?
3. Does the client end the program and print the correct result when a full word is guessed?
4. Does the server stop running when the client sends the QUIT command?



Task 2: Battleship (Adapted from YIJC prelim 2020 P2/Q1)

You are to write a server program for an asymmetric Battleship game, for a given client program. In this game of Battleship, there are 3 ships within a 12 metres by 8 metres region. The region is represented by a grid and each square metre has an x-coordinate and a y-coordinate. The top left square metre of the region display has $x = 0$ and $y = 0$.

Each square metre of a region is represented by a dot “.” and each square metre of a ship is represented by an “S”.

In one of the games, 3 ships were positioned in the region given in the text file **GAME.TXT**. The data in the file is read by the **server** program and stored in a list of tuples, which consists of x and y coordinates of the ships. For example, given the following data from the text file, the list of tuples read into the program is [(9, 1), (1, 2), (2, 2), (3, 2), (4, 2), (9, 2), (9, 3), (9, 4), (9, 5), (3, 6), (4, 6), (5, 6)].

.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	S	.	.	.	.	.
.	S	S	S	S	.	.	.	.	S	.	.	.	.	.
.	.	.	.	.	.	.	.	.	S	.	.	.	.	.
.	.	.	.	.	.	.	.	.	S	.	.	.	.	.
.	.	.	.	.	.	.	.	.	S	.	.	.	.	.
.	.	.	S	S	S	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.

The **server** program will send a message to inform player (on **client** side) that game has started.

On the **client** program, the player only sees the dots but not the 3 ships in the region.

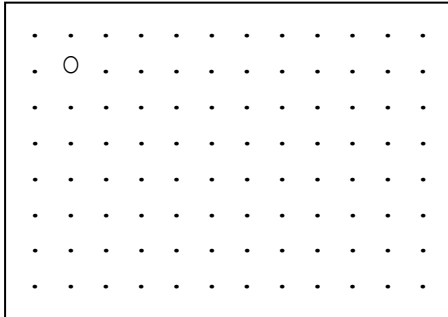
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.

The player is asked to fire 7 missiles targeting at specific locations, one at a time, and the outcome displayed after each missile shot. The input will be an x and y coordinate for each location, to be sent to the server.

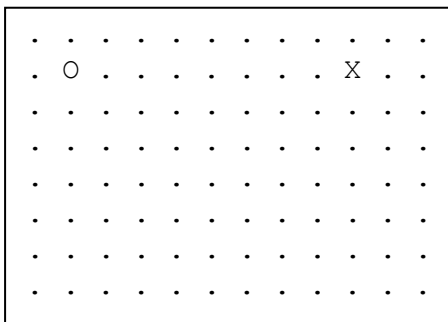
If the missile strikes any part of a ship, the “S” will be changed to an “X”. If the missile missed the ship, the “.” will be changed to an “O”.



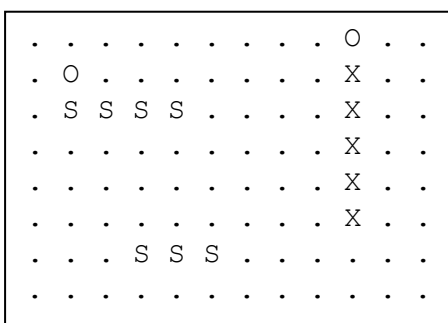
For example, when the player fires a missile at (1,1). Since the player missed the ship, the ‘.’ will change to ‘O’ and the region will be displayed as:



In the next try, suppose the player fires a missile at (9,1). Since a part of the ship is hit, the ‘S’ will change to ‘X’ and the region will be displayed as:



At the end of 7 tries, the final region should be displayed on **client** side for player to see. There should be a total of 7 ‘O’s or ‘X’s that represent 7 misses or hits.



For each missile that the player (**client** side) fires, the x and y coordinate will be sent to the **server** program. The **server** program will update its data to reflect whether a ship is hit or missed. It will also send the region data to the **client** to display for player to see whether a ship is hit or missed after each shot.

Work in pairs to validate each other’s programs. One of you will act as the server host and the other as client.